

EKF #2 Stationary-Drift Debug Journey

Multi-Sensor Fusion Pipeline — Husarion Lynx (Gazebo Harmonic)

Anish Kumar, Sachit Bansal, Somanshu Juneja
Under guidance of Dr. Radhe Shyam

2026-05-09

Abstract

This document records every fix attempted on the global EKF (`ekf_filter_node_map`) during the resolution of a stationary-drift bug. The robot, while not moving, was reported by EKF #2 at positions up to 21 m away from ground truth. Fifteen separate hypotheses were tested. The final root cause was a **circular reference loop**: `navsat_transform` was fed by EKF #1 (odom frame), so its `/odometry/gps` output lived in the odom frame. EKF #2 then transformed those measurements through the `map→odom` TF that EKF #2 itself publishes from its own state, producing self-consistent garbage that never corrected the state. Switching `navsat`'s input to EKF #2's output broke the loop and the system locked to RTK ground truth within 1–3 cm.

1 Problem Statement

Robot stationary at world origin in Gazebo (ground truth: $x = y = 0$). Topics inspected:

- `/odometry/local` (EKF #1, odom frame, no GPS): drift ≤ 5 mm ✓
- `/odometry/global` (EKF #2, map frame, with GPS): drift up to **21 m** in 100 s
- `/odometry/gps` (`navsat` output): drift ≤ 2 cm ✓
- Raw GPS `/gps/fix`: drift ≤ 2.3 cm ✓

All upstream sensors clean. Bug isolated to EKF #2 fusion math.

2 Diagnostic Infrastructure

A Jupyter notebook `notebooks/pipeline_drift_debug.ipynb` was built to extract every pipeline stage from a stationary rosbag and quantify per-stage drift. Eleven sections:

1. Topic rates / sample counts
2. Raw `/gps/fix` scatter (Gazebo NavSat plugin direct output)
3. `/gps/fix/covariance_fixed` relay pass-through verification
4. `/odometry/gps` `navsat`-projected scatter
5. Visual + LiDAR odometry stationary noise per-frame deltas
6. EKF #1 stationary drift
7. EKF #2 stationary drift
8. Pipeline overlay (XY , $X(t)$, $Y(t)$, $\text{yaw}(t)$) on one figure
9. IMU bias check (gyro $Z \rightarrow$ yaw drift, accel $X/Y \rightarrow$ position drift via $\frac{1}{2}bt^2$)
10. EKF #2 GPS innovation analysis
11. Per-stage drift attribution table + auto-flag heuristics

Companion script `record_stationary_pipeline.sh` captures all topics needed for the analysis with one bag command.

3 Fix Attempts — Chronological

Each row: hypothesis → change → result. Status legend: ✓=fixed it, ~~=helped partially, ✗=no improvement or made worse.

3.1 Fix 1: URDF GPS noise too high (consumer-grade)

Hypothesis: Gazebo NavSat scatter $\sigma \approx 0.5$ m too noisy for 10 Hz fusion.

Change: URDF stddev: 4.5×10^{-6} deg → 9×10^{-8} deg (~ 1 cm RTK).

Status: ✓ Raw GPS scatter dropped to 1 cm. Relay variance updated to match (1×10^{-4} m²). Necessary baseline but did not fix EKF #2 drift.

3.2 Fix 2: IMU accelerometer bias mean = 0.1

Hypothesis: Notebook measured accel X/Y bias $\approx +0.08$ m/s². URDF check revealed `<bias_mean>0.1</bias_mean>` on all three accel axes.

Change: bias_mean: 0.1 → 0.0, bias_stddev: 0.001 → 0.0001.

Math: Predicted drift $\Delta x = \frac{1}{2} \cdot 0.1 \cdot 158^2 \approx 1248$ m without correction; in EKF integration manifested as *velocity* drift via accel double-integration.

Status: ✓ IMU accel bias dropped from +0.08 to +0.001 m/s².

3.3 Fix 3: navsat frequency 1 Hz → 10 Hz

Hypothesis: Match GPS sensor rate so EKF #2 sees full RTK update cadence.

Status: ✓ Necessary; works with RTK noise (would have caused jitter at $\sigma = 0.5$ m).

3.4 Fix 4: Tighten EKF #2 process noise on v_y

Hypothesis: Diff-drive non-holonomic: $v_y \equiv 0$. Loose $Q[v_y] = 0.05$ lets state walk.

Change: $Q[v_x] = Q[v_y] : 0.05 \rightarrow 0.001$.

Status: ~ Position drift dropped 15 m → 7 m, but yaw drift WORSENERD $-0.75^\circ \rightarrow -15.8^\circ$. Error pumping: locked v_y pushed mismatch into yaw via $P[v_y, \theta]$ cross-covariance.

3.5 Fix 5: Disable IMU linear acceleration in EKF #2

Hypothesis: Even with bias_mean=0, residual accel noise integrates to velocity. Wheel and GPS already constrain velocity.

Change: imu0_config[12..14]: true,true,true → false,false,false.

Status: ~ Drift 7 m → 4.5 m. Accel was indeed double-integrating residual bias.

3.6 Fix 6: Switch VO/LO from position-fusion to velocity-fusion

Hypothesis: VO/LO in differential=true mode summed thousand position deltas → random walk drift accumulates. Velocity fusion lets EKF integrate cleanly.

Change: odom1/odom2_config: position rows OFF, velocity rows ON; differential=false.

Status: ✗ Worse: yaw -22.7° , drift 5.21 m. Visual odom publishes velocity with *artificially tight covariance* (cov = 3.4×10^{-6} , $\sigma \approx 2$ mm/s) so the filter overweights its biased v_x reading by a factor of ~ 300 over wheel.

3.7 Fix 7: Tighten yaw and v_{yaw} process noise

Hypothesis: Loose $Q[\theta] = 0.06$, $Q[\omega_z] = 0.02$ let GPS x,y updates drag yaw via cross-covariance.

Change: Both → 0.001.

Status: ✓ for yaw (locked at $\sim 0.1^\circ$); did not fix position.

3.8 Fix 8: Set imu0_differential=false in EKF #2

Status: No-op (orientation rows were already false). Cleanup only.

3.9 Fix 9: Drop VO/LO entirely from EKF #2

Hypothesis: Their tight twist covariance is unfixable; lean fusion is more robust.

Change: EKF #2 = wheel + IMU + GPS only.

Status: ✗ Yaw walked -238° in 58 s. Removing LiDAR removed the only absolute yaw anchor; gyro Z bias integrated unchecked.

3.10 Fix 10: Increase $Q[x, y]$ to 0.5

Hypothesis: Tight $Q[x, y] = 0.025$ keeps P_{xx} small; once state diverges, GPS gate (3σ of $\sqrt{P+R}$) rejects updates.

Change: $Q[x, y] : 0.025 \rightarrow 0.5$.

Status: ✗ Made initial transient worse: P explodes during 18 s navsat startup gap, state runs.

3.11 Fix 11: Compromise — $Q[x, y] = 0.05$

Status: Acceptable steady-state but not the cure.

3.12 Fix 12: GPS rejection threshold 3.0 → 100

Hypothesis: With RTK $\sigma = 1$ cm, no realistic outliers possible; trust always.

Status: ✓ Necessary; without it the diverged state self-locks because GPS gets rejected.

3.13 Fix 13: Enable wheel v_y measurement (odom0_config[7]=true)

Hypothesis: Inspecting EKF #2 internal state via rostopic echo revealed $v_y = +7.33$ m/s while body twist read near zero in output — v_y state had no measurement source (wheel didn't report it, IMU accel disabled, GPS no velocity).

Change: odom0_config[7]: false → true. diff_drive_controller publishes $v_y = 0$ with cov 0.001 as a kinematic constraint.

Status: ✓ v_y state locked. Predicted Y-runaway eliminated. Drift dropped to ~ 3 m steady-state offset (still wrong, but no longer growing).

3.14 Fix 14: LiDAR yaw absolute in differential mode

Hypothesis: EKF #2 has no absolute yaw measurement; only velocity sources for yaw rate. Even with tight $Q[v_{yaw}]$, integrated yaw drifts at $0.16^\circ/\text{s}$.

Change: Add odom2: /odometry/lidar with *only* yaw row 5 enabled and differential=true (uses ICP $\Delta\theta$ between samples, immune to absolute offset).

Status: ✓ Yaw locked at $\leq 0.1^\circ$ steady. But position still 2.7 m off.

3.15 Fix 15: Circular Reference Loop — the actual root cause

After fixes 1–14, EKF #2 had:

- velocity locked at $\leq 10^{-5}$ m/s (verified via internal state inspection)
- yaw locked at $\leq 0.1^\circ$
- TF map→odom showing $(-21.3, -1.4)$ m
- GPS reporting $(0, 0)$ in odom frame
- EKF #2 not snapping to GPS despite all gates accepting it

The only remaining explanation: **GPS measurements were not actually correcting state.**

4 The Circular Reference Bug

4.1 Topology Before Fix

Broken topology

```
EKF#1 (world_frame=odom)    navsat_transform

                               /odometry/gps  (frame_id="odom")

                               EKF#2  (world_frame=map)
                               uses TF map→odom to transform

                               tf2 buffer

                               publishes map→odom
                               based on its own state
```

Each step in the chain:

1. navsat reads EKF #1's continuous odometry (/odometry/local, frame=odom).
2. navsat converts incoming GPS lat/lon → XY using EKF #1's yaw and the locked datum. Output /odometry/gps carries frame_id=odom, since navsat copies its input odometry's frame.
3. EKF #2 reads /odometry/gps. Its world frame is map. To use a measurement tagged odom, it queries TF for map→odom.
4. EKF #2 *itself* publishes the map→odom transform every cycle, computed as (EKF#2_state_in_map) – (EKF#1_state_in_odom).

4.2 Why It Self-Consistently Fails

For a stationary robot:

$$\begin{aligned} T_{\text{map} \rightarrow \text{odom}}(t) &= P_{\text{EKF}\#2}(t) - P_{\text{EKF}\#1}(t) \\ &\approx P_{\text{EKF}\#2}(t) \quad (\text{since EKF}\#1 \text{ holds } 0) \end{aligned}$$

$$\text{GPS_in_odom} \approx 0 \quad (\text{robot at datum, stationary})$$

$$\text{GPS_in_map} = T_{\text{map} \rightarrow \text{odom}} \cdot \text{GPS_in_odom} = P_{\text{EKF}\#2}(t)$$

The GPS measurement, after frame transformation, *equals the EKF's own state*. The innovation $v = z - H\hat{x}$ is identically zero. **No correction is ever applied.** Once the state has drifted (e.g. from prior bugs in fixes 1–14), GPS cannot pull it back because GPS itself gets rotated/translated to wherever the EKF thinks it is.

4.3 Confirmation via TF Inspection

```
$ ros2 run tf2_ros tf2_echo map odom
- Translation: [-21.306, -1.416, 0.000] # EKF#2 thinks robot is here
- Rotation:    [0.000, 0.000, 0.014, 1.000]
```

EKF #2 wandered 21.3 m in map frame. /odometry/gps showed (–0.013, +0.002) in frame_id=odom. Transformed through $T_{\text{map} \rightarrow \text{odom}}$ that EKF #2 publishes itself, the GPS measurement landed at (–21.32, –1.41) in map — *exactly where EKF #2 already believed it was*. Self-consistent garbage.

4.4 The Fix — Linear Topology

Change navsat's input from EKF #1 to EKF #2:

```
# baseline_ekf.launch.py
remappings=[
    ("imu", "imu/data/covariance_fixed"),
    ("gps/fix", "gps/fix/covariance_fixed"),
    - ("odometry/filtered", "odometry/local"),
    + ("odometry/filtered", "odometry/global"),
],
```

Fixed topology

```
GPS lat/lon  navsat (uses EKF#2 yaw, locked datum)  /odometry/gps
                                                    (frame_id="map")

                                                    EKF#2  (world_frame=map)
                                                    direct fusion, no TF
                                                    transformation needed
```

Now navsat copies EKF #2's frame (map) into its output. EKF #2 reads the message in its own world frame — no TF lookup, no circular dependency. The innovation is genuine $v = z_{\text{GPS, map}} - \hat{x}_{\text{EKF\#2}}$, the Kalman gain pulls state to GPS, the system locks to ground truth.

4.5 Concern: Feedback Loop?

The original launch comment warned: “*map to local! not global. global make loop and robot go crazy.*” That warning held when EKF #2 was nonlinear in inputs (visual + LiDAR position deltas, accel integration, loose process noise). With the lean post-fix EKF #2 (wheel + IMU ω_z + LiDAR $\Delta\theta$ + GPS x, y , all velocities tightly constrained), the EKF #2→navsat→EKF #2 path is stable: GPS pulls state to truth, navsat re-projects GPS using truth-aligned yaw, no oscillation observed.

5 Final Working Configuration

5.1 EKF #2 Sensor Matrix

Sensor	Topic	Fused channels
Wheel encoder	/diff_drive_controller/odom	v_x, v_y, ω_z
LiDAR (ICP)	/odometry/lidar	yaw (differential mode, $\Delta\theta$)
IMU	/imu/data/covariance_fixed	$\omega_x, \omega_y, \omega_z$
GPS (RTK)	/odometry/gps (in map frame)	x, y

5.2 Process Noise Diagonal

State	Q value	Notes
x, y	0.05	Moderate, GPS-corrected fast
θ (yaw)	0.001	Tight; LiDAR $\Delta\theta$ anchors
v_x, v_y	0.001	Tight; non-holonomic + wheel constraint
ω_z	0.001	Tight; wheel + IMU velocity

5.3 Sensor Noise Sources

Layer	Param	Value
URDF GPS horiz stddev	—	9×10^{-8} deg (~ 1 cm)
URDF GPS vert stddev	—	2.7×10^{-7} deg (~ 3 cm)
URDF accel bias_mean	all axes	0.0
URDF accel bias_stddev	all axes	0.0001
URDF gyro bias_stddev	all axes	0.0001
Relay GPS_VAR_HORIZONTAL	—	1×10^{-4} m ²
EKF #2 GPS rejection threshold	—	100
navsat frequency	—	10 Hz

6 Verification — Stationary Hold

10-second sample of /odometry/global after all fixes:

t (s)	x (m)	y (m)	yaw (deg)
1	-0.0027	+0.0140	-0.105
3	-0.0023	+0.0165	-0.093
5	+0.0067	+0.0103	-0.105
7	-0.0008	+0.0034	-0.098
9	-0.0027	-0.0144	-0.102

Position scatter ~ 1 cm in X, ~ 3 cm in Y — matches RTK noise floor. Yaw locked within 0.01° .

TF map $\rightarrow$ odom: $(-0.020, 0.018)$ m — both frames coincident as expected for a robot at the datum.

7 Case Study — How the Circular Reference Was Found

This section reconstructs the live debugging session that exposed Fix 15. After fixes 1–14 the system still drifted by several meters. Each step below shows the exact terminal output that drove the next hypothesis.

7.1 Step 1 — Quick health check after Fix 14

```
$ for t in /odometry/global /odometry/local /odometry/gps /ground_truth/odom; do
  echo "=== $t ==="
  ros2 topic echo $t --once | grep -A 2 "position:" | head -4
done
```

```
=== /odometry/global ===
position: x: -2.7306    y: 0.8205
=== /odometry/local ===
position: x: 0.006     y: 0.0002
=== /odometry/gps ===
position: x: -0.014    y: 0.017
=== /ground_truth/odom ===
position: x: -8.6e-10  y: 9.1e-12
```

Read: EKF #1, GPS, ground truth all agree (within 2 cm). EKF #2 alone reports $(-2.73, +0.82)$ m. Drift confined to EKF #2.

7.2 Step 2 — Is EKF #2 yaw walking?

```
$ ros2 topic echo /odometry/global --once | grep -A 4 orientation
```

```
orientation:
  z: -0.0005
  w: 0.99999987
```

$\theta = 2\arcsin(0.0005) \approx -0.06^\circ$. **Yaw locked.** (Fix 14 with LiDAR $\Delta\theta$ working.)

7.3 Step 3 — Is internal velocity walking?

```
$ ros2 topic echo /odometry/global --once | grep -A 8 twist
```

```
twist:
  twist:
    linear:
      x: -1.93e-06
      y: 1.48e-05
    angular:
      z: -2.13e-04
```

Velocity locked at machine zero. $v_x \approx 10^{-6}$ m/s, $v_y \approx 10^{-5}$ m/s.

7.4 Step 4 — The contradiction

EKF #2 reports velocity = 0 but position drifting (−2.73 m on X, +0.82 m on Y, growing over time). **Mathematically impossible** under standard 2D rigid-body kinematics:

$$\dot{x} = v_x \cos \theta - v_y \sin \theta$$

With $v_x \approx v_y \approx 0$ and $\theta \approx 0$, $\dot{x}$ should also be zero.

Hypothesis: The position EKF #2 reports must be coming from somewhere other than its own velocity-integration prediction. Either the state has been initialized wrong, or GPS is being applied incorrectly.

7.5 Step 5 — Confirm GPS arriving and being processed

```
$ timeout 8 ros2 topic hz /odometry/gps
average rate: 7.083 min: 0.099s max: 0.200s
```

```
$ ros2 topic echo /odometry/gps --once | head -15
```

```
header:
  stamp: { sec: 150, nanosec: 900000000 }
  frame_id: odom <-- !!
  child_frame_id: ''
pose:
  pose:
    position: { x: -0.005, y: 0.002 }
```

GPS publishing at 7 Hz, position ≈ 0 . **But** frame_id = odom, **not** map. First red flag.

7.6 Step 6 — Are gates accepting GPS?

```
$ ros2 param get /ekf_filter_node_map odom1_pose_rejection_threshold
Double value is: 100.0
```

Threshold 100σ . With $P_{xx} = 0.004$, $\sigma = 0.064$ m. Gap from state (−2.73) to GPS (≈ 0) is 42σ — well within gate. GPS *is* being accepted.

7.7 Step 7 — Then why does state not snap to GPS?

Standard Kalman update: $K = P/(P + R)$. With $P = 0.004$ and $R = 10^{-4}$, $K = 0.975$. Each GPS update should pull state by 97.5% toward GPS. After 5 updates (< 1 s), state should be at GPS. But state stays at −2.73 m.

Hypothesis: The Kalman gain math IS being applied, but the GPS measurement that EKF #2 sees is *not actually* $(-0.005, 0.002)$. Something is transforming it.

7.8 Step 8 — Inspect TF tree (the breakthrough)

```
$ ros2 run tf2_ros tf2_echo map odom
```

```
At time 175.730000000
- Translation: [-21.306, -1.416, 0.000]
- Rotation:    [0.000, 0.000, 0.007, 1.000]
- RPY (deg):   [0.000, -0.000, 0.798]
```

```
$ ros2 run tf2_ros tf2_echo odom base_link
```

```
At time 177.385000000
- Translation: [0.011, -0.002, 0.000]
- Rotation:    [0.000, 0.000, -0.011, 1.000]
```

The smoking gun.

- odom→base_link = (0.01,0) m ✓ (EKF #1 says robot at origin in odom frame)
- map→odom = (-21.3,-1.4) m ✗ (EKF #2 says odom is shifted 21m relative to map)

EKF #2 has been silently publishing a giant offset transform between map and odom. map→base_link composes to $(-21.3, -1.4)$ m — exactly the EKF #2 internal state.

7.9 Step 9 — The math of the loop

navsat reads EKF #1 (frame=odom). Its output /odometry/gps carries frame_id=odom too (navsat copies input frame). When EKF #2 ingests this measurement, it must transform from odom into its own world frame (map):

$$z_{\text{map}} = T_{\text{map} \rightarrow \text{odom}} \cdot z_{\text{odom}}$$

But $T_{\text{map} \rightarrow \text{odom}}$ is published by EKF #2 itself, computed as

$$T_{\text{map} \rightarrow \text{odom}}(t) = \hat{x}_{\text{EKF}\#2}(t) - \hat{x}_{\text{EKF}\#1}(t)$$

With EKF #1 holding origin, $T_{\text{map} \rightarrow \text{odom}} = \hat{x}_{\text{EKF}\#2}$. And GPS in odom ≈ 0 . Substituting:

$$z_{\text{map}} = \hat{x}_{\text{EKF}\#2} \cdot 0 + \hat{x}_{\text{EKF}\#2} = \hat{x}_{\text{EKF}\#2}$$

The transformed GPS measurement equals EKF #2's own state estimate. Innovation $v = z - H\hat{x} = 0$ identically. Kalman gain is irrelevant; with zero innovation, no correction is ever applied regardless of K .

7.10 Step 10 — Verify with numbers

```
GPS in odom frame:      ( -0.005,  0.002)
T_map->odom (EKF#2):    (-21.306, -1.416)

GPS transformed to map: (-21.306 + (-0.005)*cos(0.8 ),
                        -1.416  + ( 0.002)*cos(0.8 ))
                        (-21.311, -1.414)

EKF#2 internal state:   (-21.306, -1.416)  <-- same!

Innovation in map:      ( -0.005,  0.002)  <-- pre-transform residual

Kalman gain in map:     K      0.975
Correction applied:      0.975 * 0.005      5 mm

Drift accumulated by EKF prediction since last update: easily > 5 mm
```

GPS “correction” shrinks to a few mm in map frame — never enough to overcome the drift caused by tiny biases in wheel/IMU between updates. State runs away while the filter remains internally consistent.

7.11 Step 11 — The fix and the proof

Change navsat's input from EKF #1 to EKF #2:

```
# baseline_ekf.launch.py, navsat node remappings:
- ("odometry/filtered", "odometry/local"),
+ ("odometry/filtered", "odometry/global"),
```

Now navsat output /odometry/gps carries frame_id=map. EKF #2 reads the message in its own world frame, no TF transformation, the innovation is genuine: $\mathbf{v} = \mathbf{z}_{\text{GPS, map}} - \hat{\mathbf{x}}_{\text{EKF\#2}}$.

After restart:

```
$ ros2 run tf2_ros tf2_echo map odom
- Translation: [-0.020, 0.018, 0.000] <-- 21m drop to 2cm

$ ros2 topic echo /odometry/global --once | grep -A 2 position
position: x: -0.0028 y: 0.0140 <-- locked to ground truth
```

21 m of drift dissolves into 2 cm RTK noise. The bug was not in the filter, the noise model, the process covariance, the rejection thresholds, the sensor selection, the kinematic constraints, or the state vector. It was in *which transform was being looked up* — and that transform was being computed by the very node that needed it.

7.12 Why It Took 14 Other Fixes To Find

- Most fixes were *still necessary*. Without IMU bias zero, vy locking, yaw process noise tightening, and a yaw anchor, state would have drifted independently of the circular reference. Even after Fix 15, omitting any earlier fix re-introduces the corresponding failure mode.
- The circular reference is invisible in any single topic — both /odometry/gps and /odometry/global look reasonable in isolation. Only tf2_echo on map→odom reveals the offset.
- The original launch comment (“*map to local! not global. global make loop and robot go crazy*”) actively discouraged the correct topology because it had been tried when EKF #2 was nonlinear with VO/LO position fusion — back then the feedback was unstable.
- Innovation analysis (notebook section 9) reported RMSE around 1–5 m, but did not flag the loop because the analysis used the *published* GPS in odom frame, not the post-transform GPS in map frame that EKF #2 actually saw.

8 Lessons Learned

1. **Live state inspection beats static analysis.** Multiple iterations chased process-noise tuning and rejection thresholds. The actual bug only surfaced via direct `rostopic echo /odometry/global` which revealed a hidden internal $v_y = 7.33$ m/s, and then a `tf2_echo map odom` which exposed the 21.3 m mismatch invisible from any single topic alone.
2. **Source-of-truth coverage matters.** EKF #2 originally had no absolute yaw measurement and no v_y measurement. Both gaps caused state runaway. Adding LiDAR $\Delta\theta$ fixed yaw, enabling wheel $v_y = 0$ fusion fixed lateral velocity.
3. **Reported covariances lie.** RTAB-Map visual odometry publishes velocity covariance 3×10^{-6} ($\sigma \approx 2$ mm/s), while measured stationary noise is $\sigma \approx 15$ mm/s. A $50\times$ over-confident covariance dominates the Kalman fusion. Fix at source (relay) or drop the sensor.
4. **Frame topology is part of the algorithm.** The EKF math was correct throughout fixes 1–14. The bug was in how three nodes (navsat, EKF #1, EKF #2) referenced each other through TF. Decoupling the loop required no math change, only an input remapping.
5. **Diff-drive non-holonomic constraint is not free.** `robot_localization` does not encode it; the user must explicitly fuse $v_y = 0$ from the wheel odometry (which already publishes it) by enabling `odom0_config[7]`.

9 Diagnostic Commands — What We Ran, What It Gave Us

This section records the actual ROS 2 / shell commands used during the 15-fix journey, grouped by what insight each produced. These are the exact commands that exposed each bug; the entire debug chain is reproducible by replaying them.

9.1 Topic-level state inspection

```
# Compare every pipeline stage at one instant
for t in /odometry/global /odometry/local /odometry/gps /ground_truth/odom; do
  echo "=== $t ==="
  timeout 2 ros2 topic echo $t --once 2>/dev/null | grep -A 2 "position:" | head
  -4
done
```

What it gave: Side-by-side x/y at each stage. Showed EKF #1 at ~ 0 , GPS at ~ 0 , EKF #2 at -21 m — isolated the bug to EKF #2 fusion (not sensors).

```
# Sample EKF#2 over time to confirm active drift vs frozen offset
for i in 1 2 3 4 5; do
  POS=$(timeout 2 ros2 topic echo /odometry/global --once 2>/dev/null | \
    python3 -c "import sys,re; t=sys.stdin.read(); \
      x=re.search(r'x: (-?[0-9.e-]+)',t); \
      y=re.search(r'y: (-?[0-9.e-]+)',t); \
      print(f'{x.group(1)} {y.group(1)}')")
  echo "t=$i: $POS"; sleep 2
done
```

What it gave: Y went $38 \rightarrow 42 \rightarrow 56$ m over 8 s while body twist read ~ 0 . Logical contradiction triggered the deeper investigation.

9.2 Internal EKF state (twist field reveals hidden velocity)

```
# Full odometry message including twist (body-frame velocity in EKF state)
ros2 topic echo /odometry/global --once
```

What it gave: The decisive output — `twist.linear.y = +7.33` m/s on a stationary robot. Proved v_y state was unconstrained; led directly to Fix 13 (enable wheel v_y row).

9.3 Per-sensor message + covariance audit

```
ros2 topic echo /diff_drive_controller/odom --once
ros2 topic echo /odometry/visual --once
ros2 topic echo /odometry/lidar --once
ros2 topic echo /imu/data/covariance_fixed --once
```

What it gave: Visual odom `twist.linear.x = -0.006` m/s with `covariance[0] = 3.4e-6` ($\sigma \approx 2$ mm/s). Compared to wheel `covariance[0] = 0.001` ($\sigma \approx 32$ mm/s), visual is $18 \times$ tighter and dominated fusion $300 \times$ over wheel. This explained why Fix 6 (velocity-fusion pattern) made things worse instead of better.

9.4 Live parameter verification (config really loaded?)

```
ros2 param get /ekf_filter_node_map odom1_pose_rejection_threshold
ros2 param get /ekf_filter_node_map odom0_config
ros2 param get /ekf_filter_node_map process_noise_covariance
ros2 param get /ekf_filter_node_map sensor_timeout
ros2 param get /ekf_filter_node_map initial_estimate_covariance
```

What it gave: Confirmed yaml edits propagated to the running node and ruled out “stale config” as a culprit. Without this we would have re-edited yaml chasing a bug that was already fixed in the file but not in the live node.

9.5 Subscription audit (is the node actually fusing what we think?)

```
ros2 node info /ekf_filter_node_map | grep -A 10 Subscribers
```

What it gave: Confirmed exact set of input topics. Caught the moment when removing VO/LO from yaml succeeded (subscriptions dropped), and verified GPS subscription remained. Without this, hard to tell if a config change took effect.

9.6 TF tree inspection — the breakthrough

```
ros2 run tf2_ros tf2_echo map odom
ros2 run tf2_ros tf2_echo odom base_link
```

What it gave: The single output that broke the case open — map→odom translation = (-21.306, -1.416, 0) while odom→base_link translation = (0.011, -0.002, 0). The 21 m offset existed *only* in the EKF #2-published transform. Combined with GPS reporting ~ 0 in frame_id=odom, this proved the circular reference loop: navsat output is in odom frame, EKF #2 transforms it through its own map→odom which is built from its own state → self-consistent garbage. **This single command identified the root cause after fixes 1–14 had only partially helped.**

9.7 Topic publish-rate verification

```
timeout 8 ros2 topic hz /odometry/gps # background, kill after window
timeout 8 ros2 topic hz /odometry/global
```

What it gave: GPS publishing at 7 Hz steady (close to configured 10 Hz), EKF #2 at 35 Hz. Ruled out “GPS not arriving” as the cause of EKF #2 not snapping to it.

9.8 Bag-based offline analysis

```
# Record full pipeline stationary
bash /home/anish/Multi-sensor-fusion/record_stationary_pipeline.sh

# Then in notebook:
jupyter notebook /home/anish/Multi-sensor-fusion/notebooks/pipeline_drift_debug.ipynb
```

What it gave: Per-stage drift attribution table that quantified *which* stage broke. Section 11 of the notebook auto-flagged hypotheses, e.g. “EKF #2 yaw drift -22.7° vs EKF #1 $-0.01^\circ \rightarrow P[x, \theta]$ coupling” which led to Fix 7 (tighten yaw process noise).

```
# Quick output extract from a notebook without opening Jupyter
cat notebooks/pipeline_drift_debug.ipynb | python3 -c "
import json, sys
nb = json.load(sys.stdin)
for i, c in enumerate(nb['cells']):
    if c.get('outputs'):
        text = []
        for o in c['outputs']:
            if 'text' in o: text.append(''.join(o['text']))
            if text: print(f'=== CELL {i} ==='); print(''.join(text)[:3000])
"
```

What it gave: CLI access to notebook outputs without GUI. Every “re-run notebook, post results” iteration in the debug loop was driven by this command.

9.9 Source code archaeology

```
# Find all per-sensor params robot_localization actually supports
grep -hE "declare_parameter|get_parameter" \
    src/robot_localization/src/ros_filter.cpp \
    src/robot_localization/src/navsat_transform.cpp \
    | grep -oE '[a-z_0-9]+' | sort -u

# Verify navsat-specific params
grep -rn "broadcast_cartesian_transform_as_parent_frame" \
    src/robot_localization/src/ src/robot_localization/include/
```

What it gave: Confirmed every yaml param we used is supported (no silent typos). Caught that odomN_nodelay, odomN_pose_covariance_diagonal from dual_ekf_navsat.yaml are *not* declared and silently ignored.

9.10 URDF / world / plugin survey

```
# Find every place GPS noise is configured
find src -type f \( -name "*.urdf" -o -name "*.world" -o -name "*.sdf" \) \
    | xargs grep -l -i "gps\|navsat" 2>/dev/null

grep -n -A 30 'sensor name="imu_sensor"' \
    src/husarion_ugv_description/urdf/lynx_simple.urdf

# Search for available Gazebo NavSat plugins
apt-cache search rtk gnss | grep -iE "gz|gazebo"
find /usr/lib -name "*navsat*" -o -name "*rtk*" 2>/dev/null
```

What it gave: Located $\langle \text{bias_mean} \rangle 0.1$ on accel — the second-biggest single bug after the circular reference. Also confirmed Gazebo Harmonic ships only basic NavSat (no RTK plugin); RTK was simulated via tightened URDF stddev.

9.11 Build / launch

```
# Apply yaml/urdf/launch changes
colcon build --packages-select husarion_ugv_description --symlink-install

# Verify install picked up changes
grep -E "GPS_VAR|stddev>9.0e-8|bias_mean>0.0" \
    install/husarion_ugv_description/lib/.../gps_covariance_relay.py \
    install/husarion_ugv_description/share/.../lynx_simple.urdf

# Restart everything
ros2 launch husarion_ugv_description baseline_ekf.launch.py
```

What it gave: The install/ grep was crucial during early fixes when relay edits weren't propagating without symlink-install — it caught the “edited source but install/ still has old values” trap.

9.12 Summary — the three commands that mattered most

1. `ros2 topic echo /odometry/global -once` (full message, not just position) → revealed $v_y = 7.33$ m/s hidden in twist field.
2. `ros2 run tf2_ros tf2_echo map odom` → exposed the 21 m TF offset that proved circular reference.
3. `ros2 param get /ekf_filter_node_map process_noise_covariance` → ruled out config-reload bugs every iteration.

The notebook produced macro-level drift attribution; live ROS 2 commands produced the micro-level state inspection that pinpointed each bug. Both halves were necessary — neither alone would have found the circular reference.

10 Files Changed

- `src/husarion_ugv_description/urdf/lynx_simple.urdf` — IMU + GPS noise
- `src/husarion_ugv_description/scripts/gps_covariance_relay.py` — relay variance
- `src/husarion_ugv_description/config/baseline_ekf.yaml` — EKF #2 fusion matrix + Q
- `src/husarion_ugv_description/launch/baseline_ekf.launch.py` — navsat input remap

Dagnostic: `notebooks/pipeline_drift_debug.ipynb`, `record_stationary_pipeline.sh`.